VeloVerse

Final Project Documentation

Bicycle education and comparison web application

Version 1 | 10 October 2026

Final implementation record

A consolidated account of the application purpose, implemented features, layered architecture, seven-table database and automated test results.

Project status: core features implemented and tested locally. Public deployment remains future work.

Based on the BikePedia planning documentation, the final VeloVerse database schema and the local project evidence.

Table of Contents

The report describes the final VeloVerse implementation. The earlier BikePedia documents provide the original requirements and design context.

1 Project Overview 3

2 Implemented Features 4

3 Architecture and Technology Stack 5

4 Seven Table Database Design 6

5 Reference Data Dictionary 7

6 Component Data Dictionary 8

7 Bicycle Data Dictionary 9

8 Automated Testing Results 10

9 Local Operation and Data Management 11

10 Project Status and Source Record 12

1 Project Overview

VeloVerse is an educational web application that helps beginners and cycling enthusiasts understand bicycles. Users can explore bicycle types, learn how components work, examine materials, browse selected manufacturers and models, and compare two bicycles using their stored specifications.

The core browsing, search, filtering and comparison features are implemented. The saved automated test reports record 97 passing tests. This document records the local application; public hosting has not been established as a completed project milestone.

Problem and purpose

Bicycle information is spread across manufacturer pages, retailers, forums and technical resources. Beginners need a clear explanation of what a part does, how it works and how bicycle designs differ. VeloVerse brings this information into a consistent set of connected pages, using a deliberately selected educational dataset.

Scope of the completed application

The application uses a responsive two-dimensional interface with an interactive anatomy diagram. Visitors do not need an account. The database supports educational content and model comparison, without orders, shopping carts, payments, retailer inventory or delivery records.

The current seed data defines ten bicycle types, five brands and six models. The ten types are Road, Mountain, Gravel, Hybrid, BMX, Fixed-Gear, Touring, Electric, Folding and Time-Trial / Triathlon. This includes the final type addition beyond the nine types mentioned in the earlier README.

Selected model collection

Brand	Models in the final seed data
Cervélo	S5
Brompton	Folding Bicycle
Colnago	V3-RS
Orbea	Avant
Canyon	Speedmax CFR; Endurace WMN CF SL Disc 8.0 Di2

Model facts are limited to information supported by the project sources. Unconfirmed specifications remain empty, and the interface explains missing values instead of inventing them. The collection is representative rather than an exhaustive bicycle catalogue.

2 Implemented Features

Requirement	Implemented behaviour
FR-01 Browse bicycle types	List and detail pages explain categories, intended uses, advantages and disadvantages; detail pages suggest related types.
FR-02 Explore components	An interactive anatomy diagram links to component pages. Parts are grouped by system and show descriptions, purpose, operation and common materials.
FR-03 Browse brands and models	Brand pages show country, website, description, specialities and linked models. Model details show specifications, intended use, features, advantages, disadvantages and a source link.
FR-04 Search bicycles	Catalogue search matches model name, brand and bicycle type. Matching ignores letter case and accents. Empty and unmatched searches have defined behaviour.
FR-05 Filter bicycles	Type, brand and frame-material filters can be combined with search. Dropdown choices come from values used by stored bicycles.
FR-06 Compare bicycles	Two different bicycles are displayed side by side. Incomplete, invalid, missing or duplicate selections produce explanatory messages.

Comparison behaviour

The comparison contains ten specification rows: brand, model, year, bicycle type, frame material, weight, wheel size, brake system, drivetrain and number of gears. Known differences are marked and counted. A row with missing information is counted separately and is not treated as a confirmed difference.

Unknown frame material appears as “Not confirmed”; other missing specifications appear as “Not specified”. Weight is shown in kilograms without unnecessary trailing zeros. The comparison is educational and does not assign a winner or a performance score.

Interface and reliability

The interface uses a dark theme, responsive styling, navigation links, content photography and fallback images. Image credits are retained where applicable. Missing records produce 404 responses, invalid filter values are ignored, and empty lists or results receive explanatory states.

The anatomy explorer is an implemented 2D feature. A rotatable 3D bicycle and animated mechanical systems remain possible future extensions.

3 Architecture and Technology Stack

VeloVerse is a layered Spring MVC application. Spring Boot runs the application and serves HTML generated by Thymeleaf. Controllers receive browser requests, services prepare application data, and repositories use Spring Data JPA to read PostgreSQL records.

Request and response flow

Browser → Controller → Service → Repository → PostgreSQL

Retrieved data returns to the controller model. Thymeleaf combines that model with the page template and returns HTML to the browser. CSS, JavaScript and local image assets provide presentation and interaction.

Layer	Responsibility
Browser and templates	Render navigation, educational pages, search controls and comparison results.
Controllers	Map GET routes, parse parameters, choose templates and return appropriate missing-record responses.
Services	Apply search and filters, retrieve related content and prepare the data required for rendering.
Repositories and entities	Map tables and relationships to Java objects and execute database retrieval operations.
PostgreSQL	Persist the seven application tables and enforce primary, foreign and unique keys.

Services use read-only transactions. The configuration disables Open Session in View, so related data needed by a page is loaded before template rendering. Constructor injection makes dependencies explicit and supports isolated web-layer tests.

Technology stack

Area	Technology and project use
Language and framework	Java 21; Spring Boot 4.1.1 as declared in pom.xml; Spring Web MVC.
Presentation	HTML, CSS, JavaScript and Thymeleaf templates.
Persistence	PostgreSQL; Spring Data JPA with Hibernate; PostgreSQL JDBC driver.
Build	Maven and the Maven Wrapper; Spring Boot Maven plugin.
Testing	JUnit, Mockito, Spring test support, MockMvc and Maven Surefire reports.
Version control	Git for local version history. GitHub appears in the original plan; a published repository is not required for the documented local state.

4 Seven Table Database Design

The relational schema separates reusable educational information from individual model specifications. It contains seven application tables. Temporary backup tables created by optional maintenance scripts are outside this application model.

Table	Purpose
bicycle_types	Educational categories and their uses, advantages and disadvantages.
materials	Reusable descriptions of materials and their characteristics.
component_systems	Groups of related bicycle parts.
components	Individual parts, their purpose and operation.
component_materials	Unique pairs linking components to common materials.
brands	Manufacturers and their descriptive information.
bicycles	Selected model specifications and educational details.

Relationships and cardinality

Parent and child	Rule
brands → bicycles	One brand may have zero or many bicycles; each bicycle requires one brand.
bicycle_types → bicycles	One type may classify zero or many bicycles; each bicycle requires one type.
materials → bicycles	One material may describe many frames; a bicycle has zero or one confirmed frame material.
component_systems → components	One system may contain many components; each component requires one system.
components ↔ materials	Many-to-many through component_materials. Each link requires one existing component and one existing material.

Keys and integrity

The six main entity tables use INTEGER primary keys generated by default as identity values. Explicit identifiers are still allowed for restoration operations. The component_materials table uses the combined component_id and material_id as its primary key, preventing duplicate links.

Entity names are unique for types, materials, systems, components and brands. Bicycle uniqueness is defined by brand_id, name and year. Because year can be NULL, this constraint alone does not reject every duplicate with an unknown year; the seed logic also checks existing brand and model names.

Foreign keys enforce valid relationships. The schema does not declare cascading deletions or positive-value checks for year, weight or gears. The following data dictionary describes the final schema, including its ALTER TABLE additions.

5 Reference Data Dictionary

PK means primary key; FK means foreign key. Required fields cannot be NULL. Identity identifiers are generated automatically when omitted from an insert. Optional fields may be NULL.

Bicycle types

Field	SQL type	Rule or meaning
type_id	INTEGER	PK; identity
name	VARCHAR(100)	Required; unique
description	TEXT	Required
best_use	TEXT	Optional; typical use
advantages	TEXT	Optional; one point per line
disadvantages	TEXT	Optional; one point per line

Materials

Field	SQL type	Rule or meaning
material_id	INTEGER	PK; identity
name	VARCHAR(100)	Required; unique
description	TEXT	Required
advantages	TEXT	Optional; material benefits
disadvantages	TEXT	Optional; material limitations

Brands

Field	SQL type	Rule or meaning
brand_id	INTEGER	PK; identity
name	VARCHAR(100)	Required; unique
country	VARCHAR(100)	Optional
description	TEXT	Optional
website	VARCHAR(255)	Optional; added in final schema
specialities	TEXT	Optional; added in final schema

The final brand design extends the original BikePedia specification with website and specialities. These fields are optional so pre-existing records remain valid.

6 Component Data Dictionary

Component systems

Field	SQL type	Rule or meaning
system_id	INTEGER	PK; identity
name	VARCHAR(100)	Required; unique
description	TEXT	Required

Systems organise parts into meaningful groups, allowing the overview and detail pages to present related components together.

Components

Field	SQL type	Rule or meaning
component_id	INTEGER	PK; identity
system_id	INTEGER	Required FK to component_systems.system_id
name	VARCHAR(100)	Required; unique
description	TEXT	Required
purpose	TEXT	Optional; what the part does
operation	TEXT	Optional; how the part works
image_url	VARCHAR(255)	Optional; image reference

Component materials

Field	SQL type	Rule or meaning
component_id	INTEGER	Required; PK member; FK to components.component_id
material_id	INTEGER	Required; PK member; FK to materials.material_id

The joint primary key is (component_id, material_id). A component can be associated with several common materials without repeating its description. A material can likewise be reused by many components.

How these records support anatomy pages

The component overview loads systems with their parts and creates a mapping from component names to record identifiers. Diagram labels use these identifiers to link to detail pages. Component detail pages include the part's system, associated materials and related components from the same system.

There is no bicycle-to-component assembly table in the final schema. Component materials describe general educational associations; they do not establish the exact bill of materials for a particular model.

7 Bicycle Data Dictionary

The bicycles table combines identifiers, optional technical specifications and educational text. Brand and type are required; an unconfirmed frame material remains NULL.

Field	SQL type	Rule or meaning
bicycle_id	INTEGER	PK; identity
brand_id	INTEGER	Required FK to brands.brand_id
type_id	INTEGER	Required FK to bicycle_types.type_id
frame_material_id	INTEGER	Optional FK to materials.material_id
name	VARCHAR(100)	Required; model name
year	INTEGER	Optional; confirmed model year
weight_kg	NUMERIC(5,2)	Optional; weight in kilograms
wheel_size	VARCHAR(30)	Optional
brake_system	VARCHAR(100)	Optional
drivetrain	VARCHAR(150)	Optional
gears	INTEGER	Optional
description	TEXT	Optional
image_url	VARCHAR(255)	Optional; image reference
intended_use	TEXT	Optional; final addition
features	TEXT	Optional; one item per line; final addition
advantages	TEXT	Optional; one item per line; final addition
disadvantages	TEXT	Optional; one item per line; final addition
source_url	VARCHAR(255)	Optional; model fact source; final addition

Final schema changes

The final script adds intended_use, features, advantages, disadvantages and source_url with ADD COLUMN IF NOT EXISTS. It also drops any existing NOT NULL restriction on frame_material_id, ensuring an older database accepts unconfirmed frame materials.

UNIQUE (brand_id, name, year) prevents duplicate non-NULL brand, model and year combinations. NUMERIC(5,2) provides five total digits with two decimal places; it is a storage precision, not a claim that every model has a measured weight.

8 Automated Testing Results

The saved Maven Surefire reports contain 97 executed tests: 97 passed, 0 failures, 0 errors and 0 skipped. These are existing local test results inspected for this report; the suite was not rerun as part of document generation.

Test suite	Tests	Result
BicycleControllerTest	20	Passed
BicycleTypeControllerTest	6	Passed
BrandControllerTest	10	Passed
CompareControllerTest	12	Passed
CompareRowTest	3	Passed
ComponentControllerTest	9	Passed
HomeControllerTest	4	Passed
MaterialControllerTest	4	Passed
BicycleRepositoryTest	10	Passed
BicycleTypeRepositoryTest	3	Passed
BrandRepositoryTest	3	Passed
ComponentRepositoryTest	3	Passed
BicycleServiceTest	9	Passed
VeloverseApplicationTests	1	Passed
Total	97	97 passed

What the suite covers

Web-layer tests cover pages, model attributes, links, missing records, empty states, filters, comparison validation, image attribution and fallback images. Search input escaping is checked. Unit tests cover search and filter logic as well as comparison differences, missing values and weight formatting.

Repository integration tests check seeded types, the five approved brands, the six models and their relationships. The application context test checks startup. Integration tests use a separate PostgreSQL database named veloverse_test and rebuild its data through test-only reset and initialization scripts.

Evidence limits

Passing tests support the behaviours exercised by the suite. They are not a percentage of code coverage, a performance benchmark, an accessibility certification or proof of public deployment. Responsive design remains supported by the implementation; comprehensive device and usability validation requires separate evidence.

9 Local Operation and Data Management

Running the application

The project requires Java 21 and PostgreSQL. From the project folder, configure the local database connection, ensure the veloverse database exists and start the application with the Maven Wrapper.

.\mvnw.cmd spring-boot:run

Once startup succeeds, the local website is available at http://localhost:8080. This address refers to the development machine and is not a public deployment URL.

Reproducing the automated tests

Create the dedicated veloverse_test PostgreSQL database and configure access through the test settings. Run the following command from the project folder:

.\mvnw.cmd test

The test initialization resets the test database before schema.sql and data.sql rebuild the dataset. The reset script includes a guard against use on a differently named database. Results are written to target/surefire-reports.

Schema initialization and content updates

Spring Boot runs schema.sql and then data.sql at startup because spring.sql.init.mode is set to always. Hibernate schema generation is disabled with spring.jpa.hibernate.ddl-auto=none. Table creation and final column additions are designed to be rerunnable.

CREATE TABLE IF NOT EXISTS does not update an existing table definition. The final ALTER TABLE statements therefore add the new fields and make frame material optional explicitly. Starter inserts avoid duplicating existing records and preserve existing content where their conflict checks apply.

Existing databases with an older brand and model collection may require the documented maintenance update. The project's DATA-SOURCES.md explains the approved collection, the update script and restoration notes. Content migration is separate from simply restarting the application.

Data quality and maintainability

Source URLs document model facts, while IMAGE-CREDITS.md records photo attribution and licensing details. Unknown values remain empty. New educational content can be added through the existing entities and relationships without changing every template.

Search currently loads the small catalogue and filters it in Java. Database-side filtering, pagination and indexing would be appropriate areas to evaluate if the dataset grows substantially.

10 Project Status and Source Record

Final implementation status

VeloVerse has progressed from the BikePedia design plan to a working local educational application with browsing, anatomy exploration, model search, combined filters and two-bicycle comparison. Its seven-table design is implemented and the saved automated reports record 97 passing tests.

Public deployment remains a future activity. Preparing hosting, production database configuration, externalised credentials and live-environment checks is outside the completed status documented here. The original plan's deployment checklist is therefore not presented as an achieved milestone.

Future development

Possible extensions include a rotatable and zoomable 3D bicycle, selectable 3D parts and animations explaining moving mechanical systems. These would build on the existing component content. They are proposals rather than implemented features.

Further work can also expand the verified model collection, strengthen accessibility and device testing, and adapt data retrieval for a larger catalogue. Such work should retain the distinction between confirmed specifications and unknown values.

Sources and reconciliation

Source	How it informed this report
BikePedia_Project_Documentation.pdf	Original purpose, functional requirements FR-01 to FR-06, quality goals, scope, architecture and future 3D plan.
Uploaded database-design(2).md	Original seven-table structure and relationships.
Final schema shared in the project chat; src/main/resources/schema.sql	Authoritative final fields, SQL types, keys and optional frame material.
README.md and DATA-SOURCES.md	Local setup, project status, current collection, source policy and test database process.
pom.xml and src/main implementation	Declared technology versions, layered structure, routes and implemented behaviour.
src/test and target/surefire-reports	Test scope and the verified total of 97 passing automated tests.

The final schema and current implementation take precedence over earlier planning statements. The report incorporates the ten types in the current seed script, the added brand and bicycle fields, and the optional frame-material relationship. It does not carry forward the original plan's future-tense features as unsupported completion claims.